

Classification Assignment Grid

Problem Statement:

A requirement from the Hospital, Management asked us to create a predictive model which will predict the Chronic Kidney Disease (CKD) based on the several parameters. The Client has provided the dataset of the same.

- 1.) Identify your problem statement
- 2.) Tell basic info about the dataset (Total number of rows, columns)
- 3.) Mention the pre-processing method if you're doing any (like converting string to number – nominal data)
- 4.) Develop a good model with good evaluation metric. You can use any machine learning algorithm; you can create many models. Finally, you have to come up with final model.
- 5.) All the research values of each algorithm should be documented. (You can make tabulation or screenshot of the results.)
- 6.) Mention your final model, justify why u have chosen the same.

CLASSIFICATION ASSIGNMENT ANSWER

1.Problem Statement and understanding:

Client wants to predict whether a client will get chronic Kidney disease based on the following parameters age, bgr, bu, sc, sod, pot, hrmo, pcv, we, rc, sg, al, su, rbc, pc, pcc, ba, htn, dm, cad, appet, pe and ane.

Client has provided the dataset. (CKD.csv)

Domain and Learning Identification:

Requirement is clear with the client. Inputs will be age, bgr, bu, sc, sod, pot, hrmo, pcv, we, rc, sg, al, su, rbc, pc, pcc, ba, htn, dm, cad, appet, pe and ane. Output will be Classification.

Data preprocessing Details:

The following categorical columns need to be converted to Numerical. sg, al, rbc, pc, pcc, ba, htn, dm, cad, appet, pe, ane and Classification.

Nominal Method conversion for the following columns: sg, pc, rbc, pcc and ba

Ordinal Method conversion for the following columns: htn, dm, cad, appet, pe, ane and Classification

Domain and Learning Identification Result:

Since both inputs and output are numerical, the Domain would be 'Machine Learning'.

As client's requirements, inputs and output are clear, it falls under 'Supervised Learning'.

As output is Categorical (Yes or No), it falls under 'Classification'

2. Basic Information about the Dataset:

The screenshot shows a JupyterLab notebook titled "GridCV RF Classificationn Assn". The code cell [4] displays the dataset as a pandas DataFrame. The output shows the first few rows of the dataset, which has 399 rows and 25 columns. The columns include age, bp, sg, al, su, rbc, pc, pcc, ba, bgr, ..., pcv, wc, rc, htn, dm, cad, appet, pe, and ane. The dataset is loaded from a CSV file named "CKD.csv".

```
[1]: import numpy as np
[2]: import pandas as pd
[3]: dataset = pd.read_csv("CKD.csv")
[4]: dataset
[5]: dataset
[6]: dataset
[7]: dataset = pd.get_dummies(dataset, dtype=int, drop_first=True)
```

	age	bp	sg	al	su	rbc	pc	pcc	ba	bgr	...	pcv	wc	rc	htn	dm	cad	appet	pe	ane
0	2.000000	76.459948	c	3.0	0.0	normal	abnormal	notpresent	notpresent	148.112676	...	38.868902	8408.191126	4.705597	no	no	no	yes	yes	no
1	3.000000	76.459948	c	2.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	34.000000	12300.000000	4.705597	no	no	no	yes	poor	no
2	4.000000	76.459948	a	1.0	0.0	normal	normal	notpresent	notpresent	99.000000	...	34.000000	8408.191126	4.705597	no	no	no	yes	poor	no
3	5.000000	76.459948	d	1.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	38.868902	8408.191126	4.705597	no	no	no	yes	poor	yes
4	5.000000	50.000000	c	0.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	36.000000	12400.000000	4.705597	no	no	no	yes	poor	yes
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
394	51.492308	70.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	219.000000	...	37.000000	9800.000000	4.400000	no	no	no	yes	poor	no
395	51.492308	70.000000	c	0.0	2.0	normal	normal	notpresent	notpresent	220.000000	...	27.000000	8408.191126	4.705597	yes	yes	no	yes	poor	yes
396	51.492308	70.000000	c	3.0	0.0	normal	normal	notpresent	notpresent	110.000000	...	26.000000	9200.000000	3.400000	yes	yes	no	poor	poor	no
397	51.492308	90.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	207.000000	...	38.868902	8408.191126	4.705597	yes	yes	no	yes	poor	yes
398	51.492308	80.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	100.000000	...	53.000000	8500.000000	4.900000	no	no	no	yes	poor	no

399 rows x 25 columns

There are 399 rows and 25 columns.

Post-Datapreprocessing, number of rows and columns are given in the below screenshot:

The screenshot shows a JupyterLab notebook titled "GridCV Naive Bayes Classification CategoricalNB Assn". The code cell [5] displays the dataset after one-hot encoding. The output shows the first few rows of the dataset, which has 399 rows and 28 columns. The columns include age, bp, al, su, bgr, bu, sc, sod, pot, hrmo, ..., pc_normal, pcc_present, ba_present, htn_yes, dm_yes, and cad. The dataset is loaded from a CSV file named "CKD.csv".

```
[5]: dataset = pd.get_dummies(dataset, dtype=int, drop_first=True)
[6]: dataset
[7]: dataset["classification_yes"].value_counts()
[7]: classification_yes
1 249
```

	age	bp	al	su	bgr	bu	sc	sod	pot	hrmo	...	pc_normal	pcc_present	ba_present	htn_yes	dm_yes	cad
0	2.000000	76.459948	3.0	0.0	148.112676	57.482105	3.077356	137.528754	4.627244	12.518156	...	0	0	0	0	0	0
1	3.000000	76.459948	2.0	0.0	148.112676	22.000000	0.700000	137.528754	4.627244	10.700000	...	1	0	0	0	0	0
2	4.000000	76.459948	1.0	0.0	99.000000	23.000000	0.600000	138.000000	4.400000	12.000000	...	1	0	0	0	0	0
3	5.000000	76.459948	1.0	0.0	148.112676	16.000000	0.700000	138.000000	3.200000	8.100000	...	1	0	0	0	0	0
4	5.000000	50.000000	0.0	0.0	148.112676	25.000000	0.600000	137.528754	4.627244	11.800000	...	1	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
394	51.492308	70.000000	0.0	0.0	219.000000	36.000000	1.300000	139.000000	3.700000	12.500000	...	1	0	0	0	0	0
395	51.492308	70.000000	0.0	2.0	220.000000	68.000000	2.800000	137.528754	4.627244	8.700000	...	1	0	0	1	1	1
396	51.492308	70.000000	3.0	0.0	110.000000	115.000000	6.000000	134.000000	2.700000	9.100000	...	1	0	0	1	1	1
397	51.492308	90.000000	0.0	0.0	207.000000	80.000000	6.800000	142.000000	5.500000	8.500000	...	1	0	0	1	1	1
398	51.492308	80.000000	0.0	0.0	100.000000	49.000000	1.000000	140.000000	5.000000	16.300000	...	1	0	0	0	0	0

399 rows x 28 columns

There are 399 rows and 28 columns.

3. Data-preprocessing

Home | GridCV Naive Bayes Classification | +

localhost:8888/notebooks/ML%20Classification/Classification%20Assignment/GridCV%20Naive%20Bayes%20Classification%20CategoricalNB%20Assn.ipynb?

JupyterLab | Python [conda env:base] | Trusted

File Edit View Run Kernel Settings Help

398 51.492308 80.000000 0.0 0.0 0.0 normal normal notpresent notpresent 100.000000 53.000000 8500.000000 4.900000 no no no yes poor no

399 rows x 25 columns

```
[5]: dataset = pd.get_dummies(dataset, dtype=int, drop_first=True)
```

```
[6]: dataset
```

	age	bp	al	su	bgr	bu	sc	sod	pot	hmo	pc_normal	pcc_present	ba_present	htn_yes	dm_yes	cad_
0	2.000000	76.459948	3.0	0.0	148.112676	57.482105	3.077356	137.528754	4.627244	12.518156	...	0	0	0	0	0
1	3.000000	76.459948	2.0	0.0	148.112676	22.000000	0.700000	137.528754	4.627244	10.700000	...	1	0	0	0	0
2	4.000000	76.459948	1.0	0.0	99.000000	23.000000	0.600000	138.000000	4.400000	12.000000	...	1	0	0	0	0
3	5.000000	76.459948	1.0	0.0	148.112676	16.000000	0.700000	138.000000	3.200000	8.100000	...	1	0	0	0	0
4	5.000000	50.000000	0.0	0.0	148.112676	25.000000	0.600000	137.528754	4.627244	11.800000	...	1	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
394	51.492308	70.000000	0.0	0.0	219.000000	36.000000	1.300000	139.000000	3.700000	12.500000	...	1	0	0	0	0
395	51.492308	70.000000	0.0	2.0	220.000000	68.000000	2.800000	137.528754	4.627244	8.700000	...	1	0	0	1	1
396	51.492308	70.000000	3.0	0.0	110.000000	115.000000	6.000000	134.000000	2.700000	9.100000	...	1	0	0	1	1
397	51.492308	90.000000	0.0	0.0	207.000000	80.000000	6.800000	142.000000	5.500000	8.500000	...	1	0	0	1	1
398	51.492308	80.000000	0.0	0.0	100.000000	49.000000	1.000000	140.000000	5.000000	16.300000	...	1	0	0	0	0

399 rows x 28 columns

```
[7]: dataset["classification_yes"].value_counts()
```

```
[7]: classification_yes
```

```
1    249
```

35°C Mostly cloudy

Search

ENG IN 14:32 30-07-2026

Home | GridCV Naive Bayes Classification | +

localhost:8888/notebooks/ML%20Classification/Classification%20Assignment/GridCV%20Naive%20Bayes%20Classification%20CategoricalNB%20Assn.ipynb?

JupyterLab | Python [conda env:base] | Trusted

File Edit View Run Kernel Settings Help

398 51.492308 80.000000 0.0 0.0 0.0 normal normal notpresent notpresent 100.000000 53.000000 8500.000000 4.900000 no no no yes poor no

399 rows x 25 columns

```
[5]: dataset = pd.get_dummies(dataset, dtype=int, drop_first=True)
```

```
[6]: dataset
```

	bu	sc	sod	pot	hmo	pc_normal	pcc_present	ba_present	htn_yes	dm_yes	cad_yes	appet_yes	pe_yes	ane_yes	classification_yes	
57.482105	3.077356	137.528754	4.627244	12.518156	...	0	0	0	0	0	0	0	1	1	0	1
22.000000	0.700000	137.528754	4.627244	10.700000	...	1	0	0	0	0	0	0	1	0	0	1
23.000000	0.600000	138.000000	4.400000	12.000000	...	1	0	0	0	0	0	0	1	0	0	1
16.000000	0.700000	138.000000	3.200000	8.100000	...	1	0	0	0	0	0	0	1	0	1	1
25.000000	0.600000	137.528754	4.627244	11.800000	...	1	0	0	0	0	0	0	1	0	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
36.000000	1.300000	139.000000	3.700000	12.500000	...	1	0	0	0	0	0	0	1	0	0	1
68.000000	2.800000	137.528754	4.627244	8.700000	...	1	0	0	1	1	0	0	1	0	1	1
115.000000	6.000000	134.000000	2.700000	9.100000	...	1	0	0	1	1	0	0	0	0	0	1
80.000000	6.800000	142.000000	5.500000	8.500000	...	1	0	0	1	1	0	0	1	0	1	1
49.000000	1.000000	140.000000	5.000000	16.300000	...	1	0	0	0	0	0	0	1	0	0	0

```
[7]: dataset["classification_yes"].value_counts()
```

```
[7]: classification_yes
```

```
1    249
```

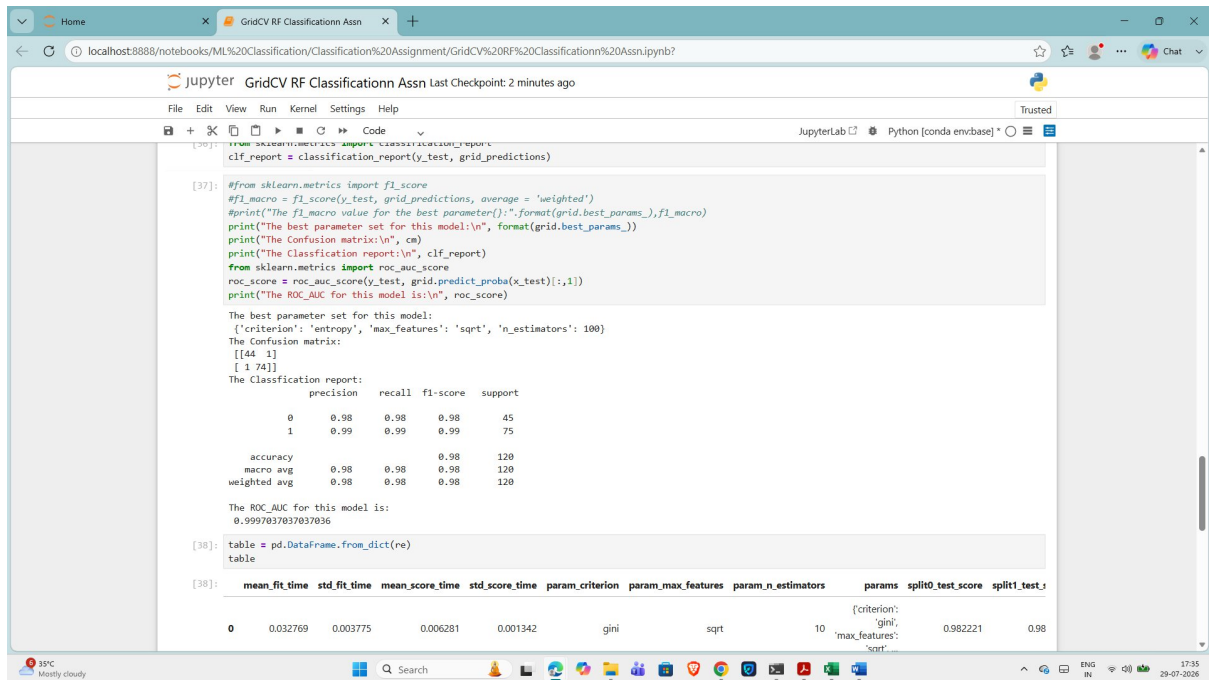
35°C Mostly cloudy

Search

ENG IN 14:32 30-07-2026

4. Creating many models to find the best model for this dataset:

4.1 Grid CV Classification Random Forest – ROC-AUC Score:



```
[37]: from sklearn.metrics import classification_report, roc_auc_score
clf_report = classification_report(y_test, grid_predictions)

[37]: #from sklearn.metrics import f1_score
#f1_macro = f1_score(y_test, grid_predictions, average = 'weighted')
#print("The f1_macro value for the best parameter():".format(grid.best_params_),f1_macro)
print("The best parameter set for this model:\n", format(grid.best_params_))
print("The Confusion matrix:\n", cm)
print("The Classification report:\n", clf_report)
from sklearn.metrics import roc_auc_score
roc_score = roc_auc_score(y_test, grid.predict_proba(x_test)[:,-1])
print("The ROC_AUC for this model is:\n", roc_score)

The best parameter set for this model:
{'criterion': 'entropy', 'max_features': 'sqrt', 'n_estimators': 100}
The Confusion matrix:
[[44  1]
 [ 1 74]]
The Classification report:
              precision    recall  f1-score   support

      0       0.98        0.98        0.98         45
      1       0.99        0.99        0.99         75

 accuracy          0.98          0.98          0.98         120
 macro avg         0.98          0.98          0.98         120
 weighted avg      0.98          0.98          0.98         120

The ROC_AUC for this model is:
0.9997037037037036

[38]: table = pd.DataFrame.from_dict(re)
table

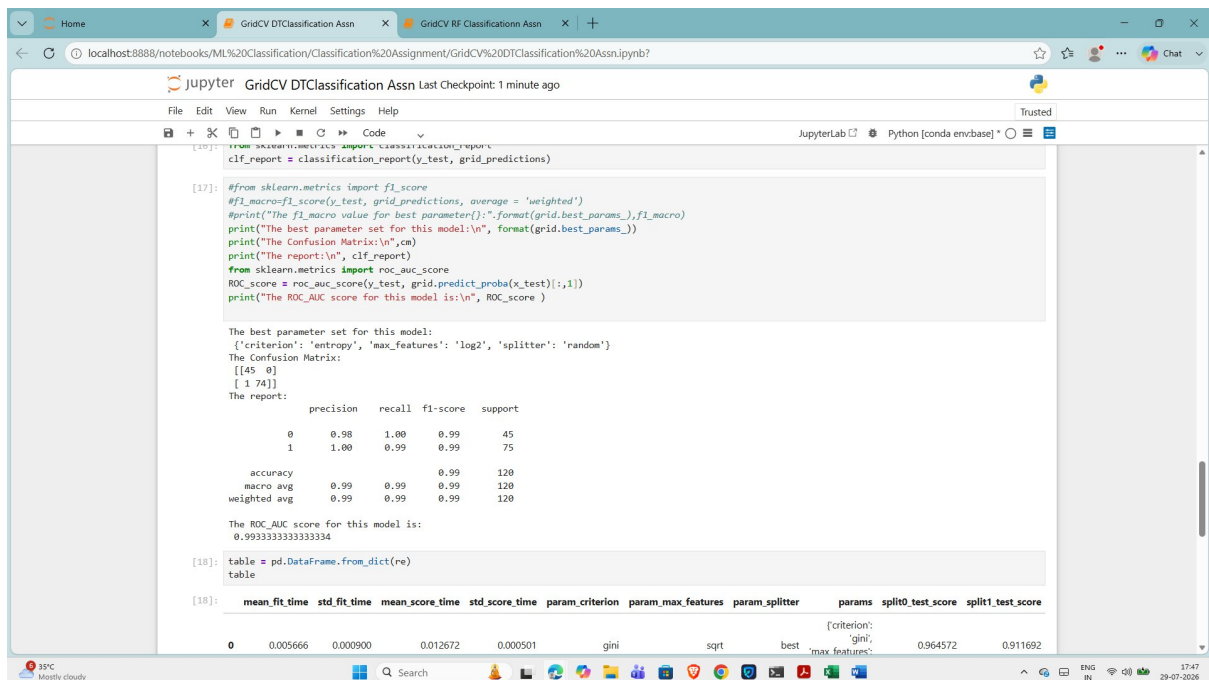
[38]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	param_n_estimators	params	split0_test_score	split1_test_score
0	0.032769	0.003775	0.006281	0.001342	gini	sqrt	10	{'criterion': 'gini', 'max_features': 'sqrt', 'n_estimators': 10}	0.982221	0.98

ROC-AUC Score for RandomForest Classification(Grid):

0.9997037037037036

4.2 Grid CV Classification Decision Tree – ROC-AUC Score:



```
[17]: from sklearn.metrics import classification_report, roc_auc_score
clf_report = classification_report(y_test, grid_predictions)

[17]: #from sklearn.metrics import f1_score
#f1_macro=f1_score(y_test, grid_predictions, average = 'weighted')
#print("The f1_macro value for best parameter():".format(grid.best_params_),f1_macro)
print("The best parameter set for this model:\n", format(grid.best_params_))
print("The Confusion Matrix:\n",cm)
print("The report:\n", clf_report)
from sklearn.metrics import roc_auc_score
ROC_score = roc_auc_score(y_test, grid.predict_proba(x_test)[:,-1])
print("The ROC_AUC score for this model is:\n", ROC_score )

The best parameter set for this model:
{'criterion': 'entropy', 'max_features': 'log2', 'splitter': 'random'}
The Confusion Matrix:
[[45  0]
 [ 1 74]]
The report:
              precision    recall  f1-score   support

      0       0.98        1.00        0.99         45
      1       1.00        0.99        0.99         75

 accuracy          0.99          0.99          0.99         120
 macro avg         0.99          0.99          0.99         120
 weighted avg      0.99          0.99          0.99         120

The ROC_AUC score for this model is:
0.9933333333333334

[18]: table = pd.DataFrame.from_dict(re)
table

[18]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	param_splitter	params	split0_test_score	split1_test_score
0	0.005666	0.000900	0.012672	0.000501	gini	sqrt	best	{'criterion': 'gini', 'max_features': 'best', 'splitter': 'random'}	0.964572	0.911692

ROC Score for DecisionTree Classification (Grid):

0.9933333333333334

4.3 Grid CV Logistic Classification – ROC-AUC Score:

```
#print("The f1_macro value for best parameter:{}:".format(grid.best_params_),f1_macro)
print("The best parameter set for this model:\n", format(grid.best_params_))
print("The Confusion Matrix:\n",cm)
print("The report:\n", ctf_report)
from sklearn.metrics import roc_auc_score
roc_score = roc_auc_score(y_test, grid.predict_proba(x_test)[:,-1])
print("The ROC_AUC score for this model is:\n", roc_score)

The best parameter set for this model:
{'penalty': 'l2', 'solver': 'newton-cg'}
The Confusion Matrix:
[[45  0]
 [ 3 72]]
The report:
      precision    recall  f1-score   support

   0       0.94       1.00       0.97        45
   1       1.00       0.96       0.98        75

 accuracy      0.97       0.97       0.97       120
 macro avg       0.97       0.98       0.97       120
weighted avg       0.98       0.97       0.98       120

The ROC_AUC score for this model is:
1.0
```

[20]: table = pd.DataFrame.from_dict(re)

[20]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_penalty	param_solver	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score
0	0.008140	0.000310	0.005152	0.001036	l2	newton-cg	{'penalty': 'l2', 'solver': 'newton-cg'}	1.000000	0.982221	0.946663	1.000000
1	0.007596	0.000697	0.004638	0.000505	l2	lbfgs	{'penalty': 'l2', 'solver': 'lbfgs'}	1.000000	0.982221	0.946663	1.000000

ROC Score for Logistic Classification (Grid):

1.0

4.4 Grid CV SVM Classification – ROC-AUC Score:

```
clf_report = classification_report(y_test, grid_predictions)

[18]: #from sklearn.metrics import f1_score
#f1_macro=f1_score(y_test, grid_predictions, average = 'weighted')
#print("The f1_macro value for best parameter:{}:".format(grid.best_params_),f1_macro)
print("The best parameter set for this model:\n", format(grid.best_params_))
print("The Confusion Matrix:\n",cm)
print("The report:\n", ctf_report)
from sklearn.metrics import roc_auc_score
roc_score = roc_auc_score(y_test, grid.predict_proba(x_test)[:,-1])
print("The ROC_Score for this model:\n", roc_score)

The best parameter set for this model:
{'C': 100, 'gamma': 'auto', 'kernel': 'rbf'}
The Confusion Matrix:
[[45  0]
 [ 4 71]]
The report:
      precision    recall  f1-score   support

   0       0.92       1.00       0.96        45
   1       1.00       0.95       0.97        75

 accuracy      0.96       0.97       0.97       120
 macro avg       0.96       0.97       0.97       120
weighted avg       0.97       0.97       0.97       120

The ROC_Score for this model:
0.9980740740740741
```

[19]: table = pd.DataFrame.from_dict(re)

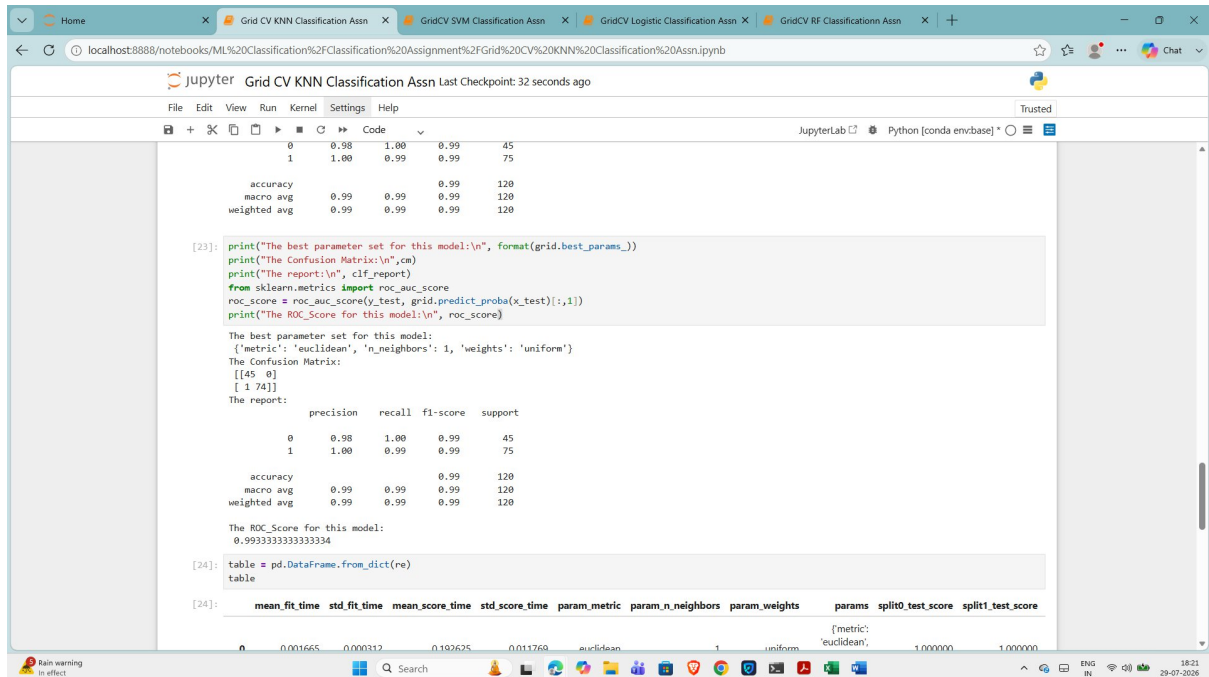
[19]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	param_gamma	param_kernel	params	split0_test_score	split1_test_score	split2_test_score
0	0.006090	0.000679	0.005153	0.000736	10	auto	linear	{'C': 10, 'gamma': 'auto', 'kernel': 'linear'}	0.982051	0.964572	0.947074

ROC Score for SVM Classification (Grid):

0.9980740740740741

4.5 Grid CV KNN Classification – ROC AUC Score:



```
[23]: print("The best parameter set for this model:\n", format(grid.best_params_))
print("The Confusion Matrix:\n", cm)
print("The report:\n", clf_report)
from sklearn.metrics import roc_auc_score
roc_score = roc_auc_score(y_test, grid.predict_proba(x_test)[:,-1])
print("The ROC_Score for this model:\n", roc_score)

The best parameter set for this model:
{'metric': 'euclidean', 'n_neighbors': 1, 'weights': 'uniform'}
The Confusion Matrix:
[[45  0]
 [ 1 74]]
The report:
      precision    recall  f1-score   support

     0       0.98       1.00       0.99         45
     1       1.00       0.99       0.99         75

 accuracy          0.99          0.99          0.99         120
 macro avg          0.99          0.99          0.99         120
 weighted avg        0.99          0.99          0.99         120

The ROC_Score for this model:
0.9933333333333334

[24]: table = pd.DataFrame.from_dict(re)
table

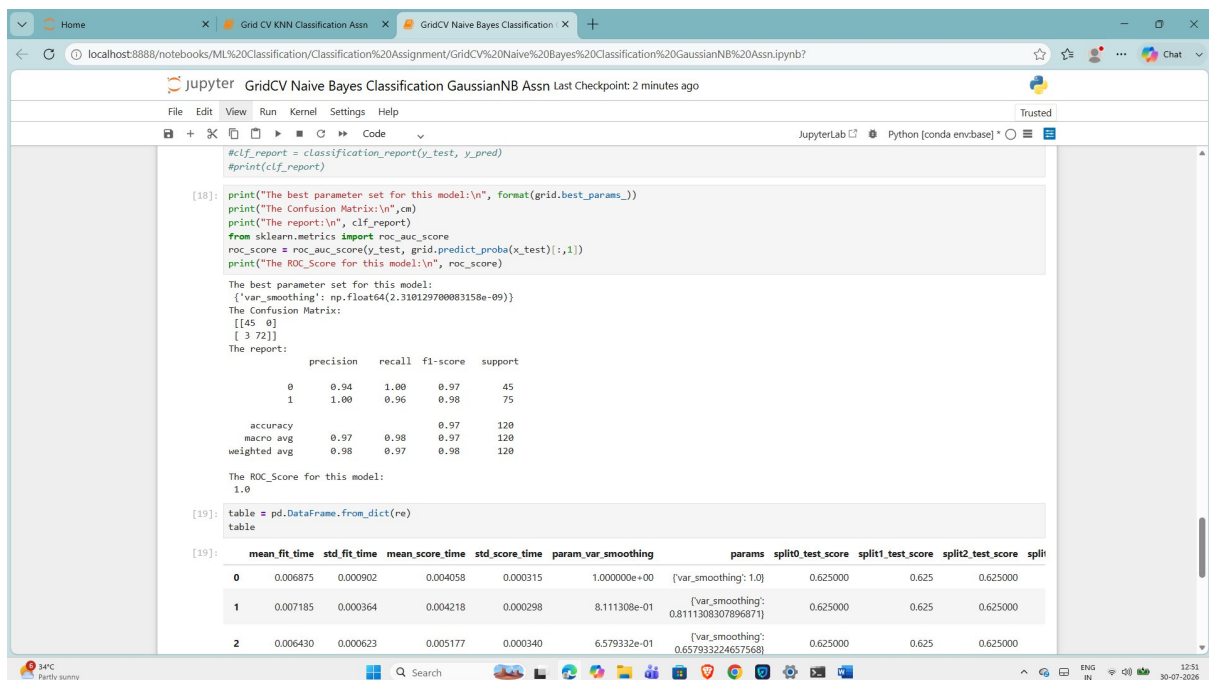
[24]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_metric	param_n_neighbors	param_weights	params	split0_test_score	split1_test_score
0	0.001665	0.000312	0.192625	0.011760	euclidean	1	uniform	{'metric': 'euclidean', 'n_neighbors': 1, 'weights': 'uniform'}	1.000000	1.000000

ROC Score for KNN Classification (Grid):

0.9933333333333334

4.6 Grid CV GaussianNB Classification – ROC AUC Score:



```
[18]: print("The best parameter set for this model:\n", format(grid.best_params_))
print("The Confusion Matrix:\n", cm)
print("The report:\n", clf_report)
from sklearn.metrics import roc_auc_score
roc_score = roc_auc_score(y_test, grid.predict_proba(x_test)[:,-1])
print("The ROC_Score for this model:\n", roc_score)

The best parameter set for this model:
{'var_smoothing': np.float64(2.310129700083158e-09)}
The Confusion Matrix:
[[45  0]
 [ 3 72]]
The report:
      precision    recall  f1-score   support

     0       0.94       1.00       0.97         45
     1       1.00       0.96       0.98         75

 accuracy          0.97          0.98          0.97         120
 macro avg          0.98          0.97          0.97         120
 weighted avg        0.98          0.97          0.98         120

The ROC_Score for this model:
1.0

[19]: table = pd.DataFrame.from_dict(re)
table

[19]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_var_smoothing	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score
0	0.006875	0.000902	0.004058	0.000315	1.000000e+00	{'var_smoothing': 1.0}	0.625000	0.625	0.625000	0.625000
1	0.007185	0.000364	0.004218	0.000298	8.111308e-01	{'var_smoothing': 0.8111308307896871}	0.625000	0.625	0.625000	0.625000
2	0.006430	0.000623	0.005177	0.000340	6.579332e-01	{'var_smoothing': 0.657933224657568}	0.625000	0.625	0.625000	0.625000

1.0

The screenshot shows a JupyterLab interface with a top bar containing file explorer and toolbar icons. The main workspace is divided into a code editor and a console. The code editor displays a Jupyter notebook cell with the following Python code:

```
[19]: print("The best parameter set for this model:\n", format(grid.best_params_))
print("The Confusion Matrix:\n", cm)
print("The report:\n", clf_report)
from sklearn.metrics import roc_auc_score
roc_score = roc_auc_score(y_test, grid.predict_proba(x_test)[:,1])
print("The ROC_Score for this model:\n", roc_score)

The best parameter set for this model:
{'alpha': np.float64(0.001), 'norm': False}

The Confusion Matrix:
[[39  6]
 [ 2 73]]

The report:
              precision    recall  f1-score   support

      0       0.95      0.87      0.91         45
      1       0.92      0.97      0.95         75

 accuracy          0.93         0.93         120
 macro avg          0.94         0.92         0.93         120
 weighted avg          0.93         0.93         0.93         120

The ROC_Score for this model:
0.9922962962962962

[20]: table = pd.DataFrame.from_dict(re)
table
```

The console output shows the results of the classification, including the best parameter set, the confusion matrix, and the ROC score. A table of results is also shown, with columns for precision, recall, f1-score, support, and various metrics.

	precision	recall	f1-score	support						
0	0.95	0.87	0.91	45						
1	0.92	0.97	0.95	75						
accuracy			0.93	120						
macro avg	0.94	0.92	0.93	120						
weighted avg	0.93	0.93	0.93	120						

The ROC_Score for this model: 0.9922962962962962

	precision	recall	f1-score	support						
8	0.005648	0.000396	0.007596	0.001474	0.006952	True	0.0069519279617756054,	0.204545	0.204545	0.204545
9	0.006251	0.000890	0.005281	0.000630	0.006952	False	0.0069519279617756054,	0.946153	0.855556	0.875
10	0.005621	0.000563	0.005655	0.000434	0.011288	True	0.011288378916846888,	0.204545	0.204545	0.204545

0.9922962962962962

```
GridCV Naive Bayes Classification Assignment/GridCV%20Naive%20Bayes%20Classification%20MultinomialNB%20Assn.ipynb

Jupyter GridCV Naive Bayes Classification MultinomialNB Assn Last Checkpoint: 7 seconds ago

File Edit View Run Kernel Settings Help Trusted

[20]: #print(cm)
      #from sklearn.metrics import classification_report
      #clf_report = classification_report(y_test, y_pred)
      #print(clf_report)

      print("The best parameter set for this model:\n", format(grid.best_params_))
      print("The Confusion Matrix:\n",cm)
      print("The report:\n", clf_report)
      from sklearn.metrics import roc_auc_score
      roc_score = roc_auc_score(y_test, grid.predict_proba(x_test)[:,:1])
      print("The ROC_Score for this model:\n", roc_score)

      The best parameter set for this model:
      {'alpha': 0.001, 'fit_prior': False}
      The Confusion Matrix:
      [[39  6]
       [ 2 73]]
      The report:

               precision    recall  f1-score   support

      0               0.95         0.87         0.91         45
      1               0.92         0.97         0.95         75

      accuracy          0.93         0.93         0.93        120
      macro avg         0.94         0.92         0.93        120
      weighted avg       0.93         0.93         0.93        120

      The ROC_Score for this model:
      0.9922962962962962

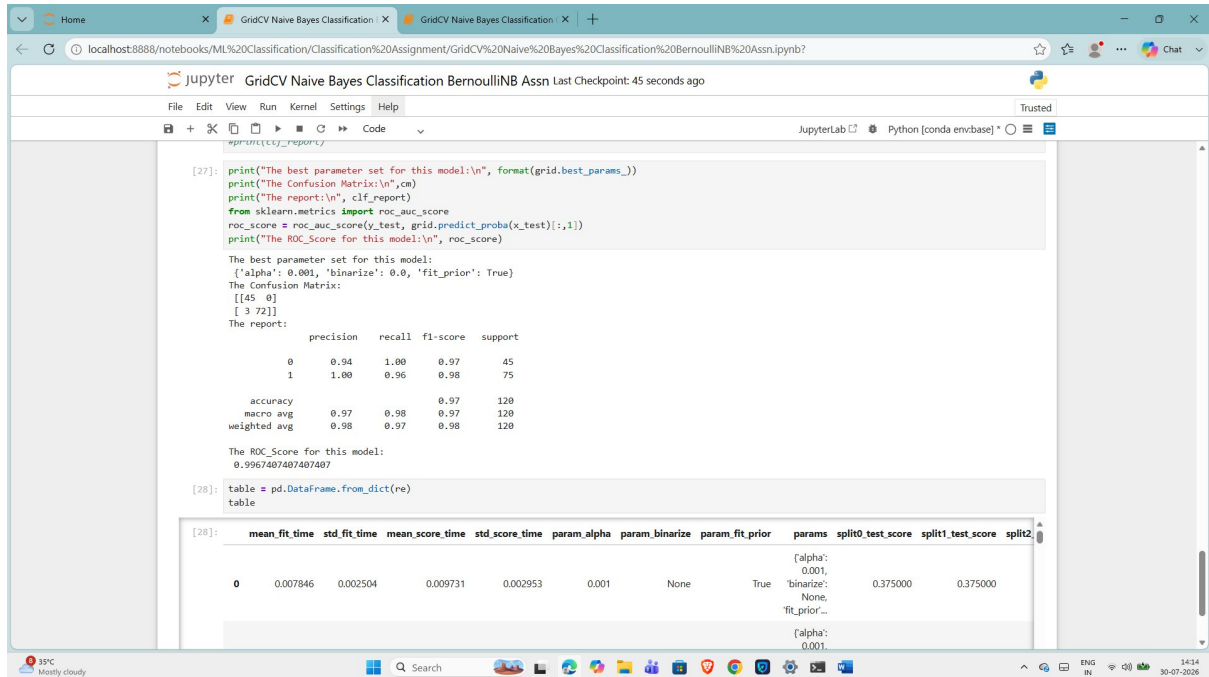
[21]: table = pd.DataFrame.from_dict(re)

[21]:   mean_fit_time  std_fit_time  mean_score_time  std_score_time  param_alpha  param_fit_prior  params  split0_test_score  split1_test_score  split2_test_score  split3_test_score
0      0.007366    0.001458      0.006304      0.001805         0.001             True  {'alpha': 0.001, 'fit_prior': True}  0.963889      0.870721      0.875548      0
```

ROC-AUC Score for MultinomialNB Classification(Grid):

0.9922962962962962

4.9 GridCV BernoulliNB Classification: ROC_AUC Score:



```
[27]: print("The best parameter set for this model:\n", format(grid.best_params_))
print("The Confusion Matrix:\n", cm)
print("The report:\n", clf_report)
from sklearn.metrics import roc_auc_score
roc_score = roc_auc_score(y_test, grid.predict_proba(x_test)[:,1])
print("The ROC_Score for this model:\n", roc_score)

The best parameter set for this model:
({'alpha': 0.001, 'binarize': 0.0, 'fit_prior': True})
The Confusion Matrix:
[[45  0]
 [ 3 72]]
The report:
              precision    recall  f1-score   support

     0       0.94      1.00      0.97        45
     1       1.00      0.96      0.98        75

 accuracy          0.97      0.97      0.97       120
 macro avg       0.97      0.98      0.97       120
 weighted avg    0.98      0.97      0.98       120

The ROC_Score for this model:
0.9967407407407407

[28]: table = pd.DataFrame.from_dict(re)
table

[28]: mean_fit_time  std_fit_time  mean_score_time  std_score_time  param_alpha  param_binarize  param_fit_prior  params  split0_test_score  split1_test_score  split2...
```

ROC-AUC Score for BernoulliNB Classification(Grid):

0.9967407407407407

ROC_AUC Score for different Models:

S.No	Classification Algorithm	ROC_AUC Score	Ranking
<u>1</u>	RandomForest Classification Grid	0.9997037037037036	<u>2</u>
<u>2</u>	DecisionTree Classification Grid	0.9933333333333334	<u>5</u>
<u>3</u>	Logistic Classification Grid	1.0	<u>1</u>
<u>4</u>	KNN Classification Grid	0.9933333333333334	<u>5</u>
<u>5</u>	SVM Classification Grid	0.9980740740740741	<u>3</u>
<u>6</u>	GaussianNB Classification Grid	1.0	<u>1</u>
<u>7</u>	ComplementNB Classification Grid	0.9922962962962962	<u>6</u>
<u>8</u>	MultinomialNB Classification Grid	0.9922962962962962	<u>6</u>
<u>9</u>	BernoulliNB Classification Grid	0.9967407407407407	<u>4</u>

Two Best Models Listed below:

1.The best model for this dataset would be '**Logistic Regression Classification Grid**', with highest **ROC_AUC score = 1**.

2.The best model for this dataset would be '**GaussianNB Classification Grid**', with highest **ROC_AUC score = 1**.